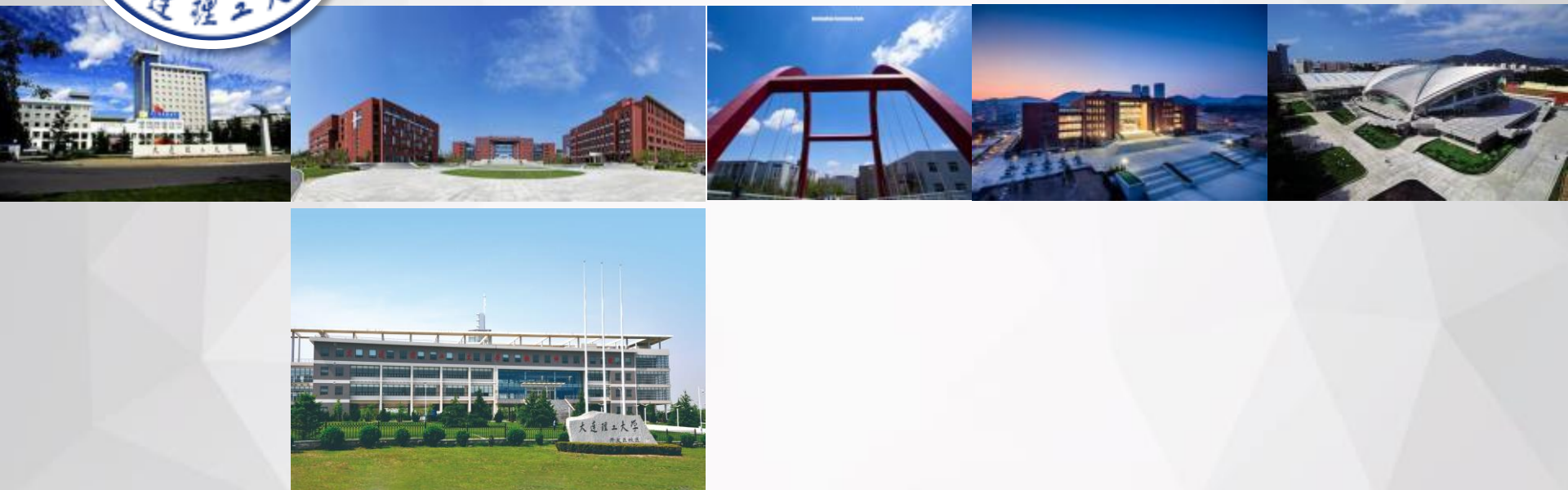




深度神经网络及训练II

DNNs and Training



01

迭代算法 (Iterative Algorithm)

02

模型泛化 (Model Generalization)

03

小结 (Summary)

PART ONE

迭代算法

Iterative Algorithm

■ 学习准则 (Learning Rule)

▶ 期望风险未知，通过经验风险近似

▶ 训练数据： $\mathcal{D} = \{x^{(n)}, y^{(n)}\}, i \in [1, N]$

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

▶ 经验风险最小化

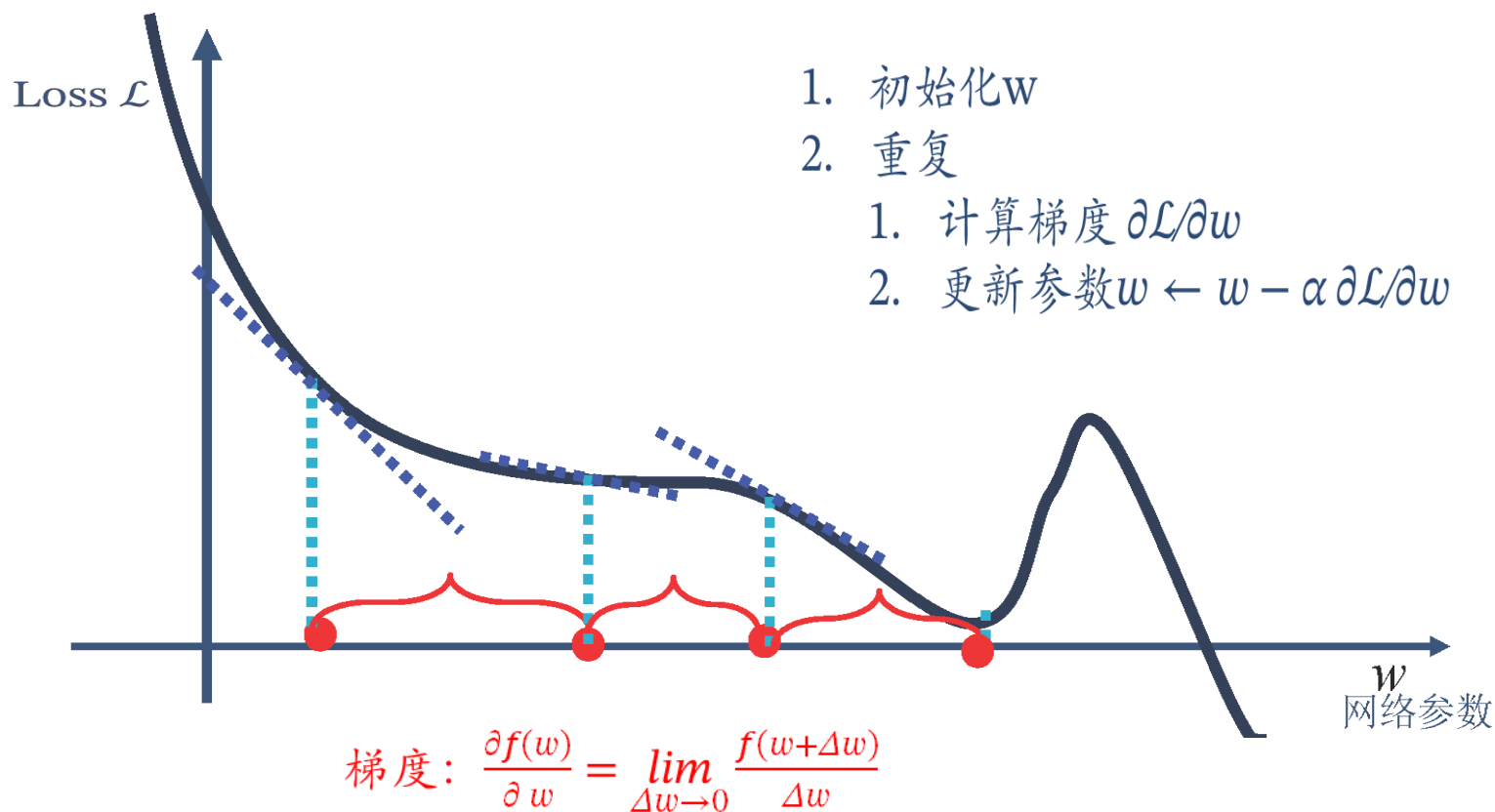
▶ 在选择合适的风险函数后，我们寻找一个参数 θ^* ，使得经验风险函数最小化。

$$\theta^* = \arg \min_{\theta} \mathcal{R}_{\mathcal{D}}^{emp}(\theta)$$

▶ 机器学习问题转化成为一个最优化问题

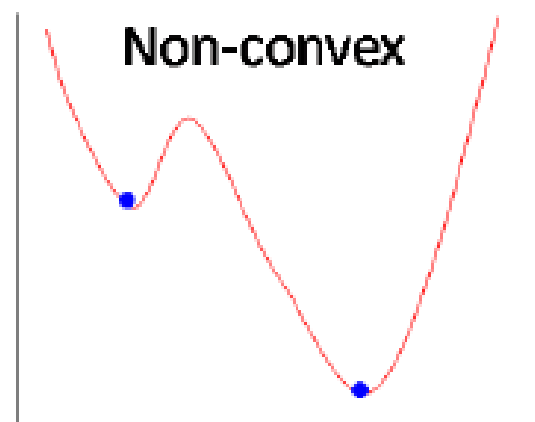
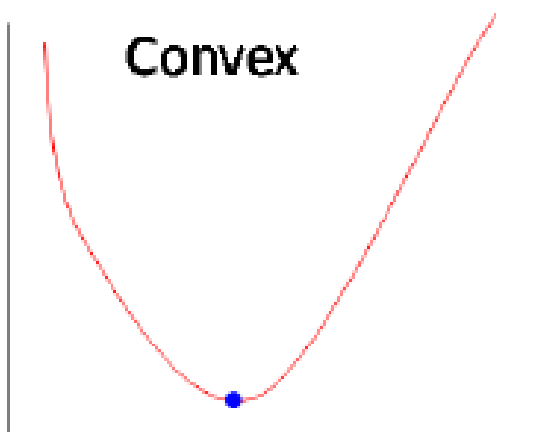
梯度下降 (Gradient Descent)

➤ 梯度下降类似盲人下山



■ 学习准则 (Learning Rule)

➤ 神经网络的学习转化为一个高阶非凸优化问题



$$\min_{\mathbf{x}} f(\mathbf{x})$$

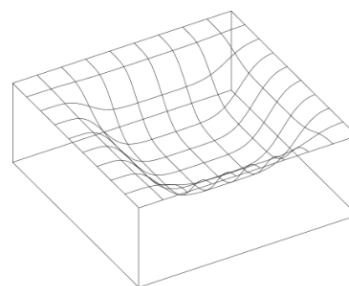
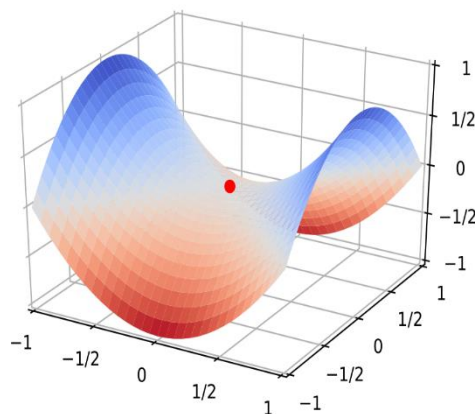
■ 难点(Hard Problems)

- 结构差异大
没有通用的优化算法
超参数多
- 非凸优化问题
参数初始化
逃离局部最优
- 梯度消失（爆炸）问题

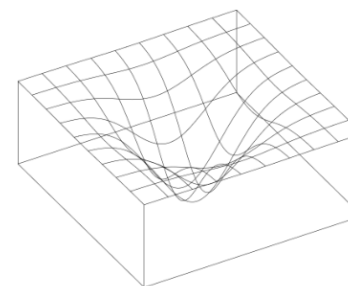
■ 非凸优化问题(Non-convex Optimal Problems)

➤ 平坦最小值问题

鞍点(Saddle Point)、驻点(Stationary Point)



(a) 平坦最小值



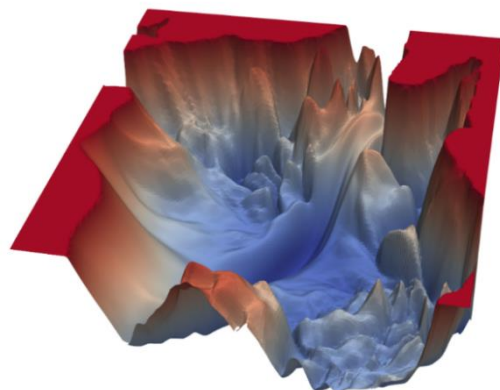
(b) 尖锐最小值

1. 梯度值为0，在平坦最小值的邻域内，所有点对应的训练损失都比较接近
2. 大部分的局部最小解是等价的
3. 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失

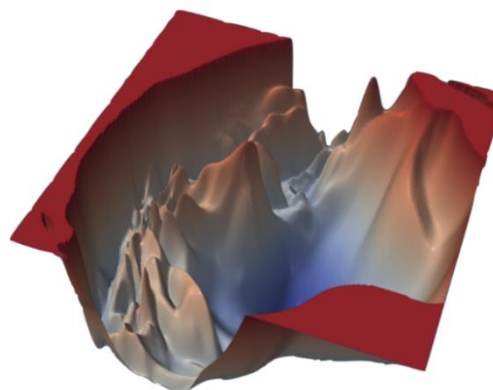
非凸优化问题(Non-convex Optimal Problems)

➤ 损失函数曲面

VGG-56

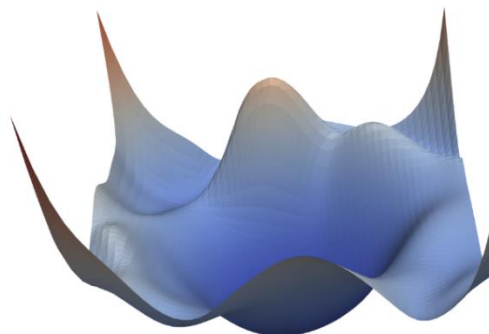


VGG-110

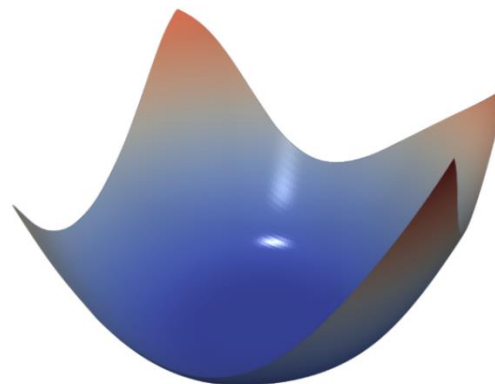


无跳跃连接

Renset-56



Densenet-121



有跳跃连接

■ 解决方案(Solutions)

- 高效的优化算法—提高效率和稳定性
动态学习率调整
梯度估计修正
- 恰当的处理方法—提高效率
参数初始化(包括超参数)
数据预处理
- 优异的网络结构—简化损失函数曲面
跳跃连接
注意力机制
激活函数
归一化机制

■ 优化算法(Optimization Methods)

➤ 梯度计算

批次梯度下降(Batch Gradient Descent)

随机梯度下降(Stochastic Gradient Descent)

小批次梯度下降(Min-Batch Gradient Descent)

➤ 更新方式

**Momentum+SGD, NAG, Adagrad, Adadelata, RMSprop
Adam, AdaMax,....**

➤ 优化算法可视化

■ 优化算法(Optimization Methods)

➤ 批次梯度下降(Batch Gradient Descent)

每次参数更新，采用**整个训练集的样本**来计算的梯度：

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta)$$

特点：

(1) 由于在一次更新中，就对整个数据集计算梯度，所以计算起来非常慢，而且不能投入新数据实时更新模型

(2) 对于凸函数可以收敛到全局极小值，对于非凸函数可以收敛到局部极小值

■ 优化算法(Optimization Methods)

➤ 随机梯度下降(Stochastic Gradient Descent)

每次参数更新，采用**训练集中的一个样本**来计算的梯度：

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)})$$

特点：

- (1) 由于在一次更新中，只针对一个样本计算梯度，所以计算起来非常快，而且投入新数据可实时更新模型
- (2) SGD的噪声样本较BGD要多，使得SGD并不是每次迭代都向着整体最优化方向。所以虽然训练速度快，但是准确度下降，并不是全局最优
- (3) SGD因为更新比较频繁，会造成损失函数有严重的震荡
- (4) BGD可以收敛到局部极小值，SGD 的震荡可能会跳到更好的局部极小值处

■ 优化算法(Optimization Methods)

➤ 小批次梯度下降(Min-Batch Gradient Descent)

每次参数更新，采用**训练集中的一个批次样本**来计算的梯度：

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)})$$

特点：

- (1) 可以降低参数更新时的方差，收敛更稳定
- (2) 可以充分地利用高度优化的矩阵操作来进行更有效的梯度计算
- (3) 不能保证很好的收敛性，学习率太小或太大都不利于学习
- (4) 批次大小的选择对结果也有较大影响(1-256)

■ 优化算法(Optimization Methods)

➤ 带动量的随机梯度下降(Momentum+SGD)

引入动量趋势，加速 SGD 来计算的梯度：

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$
$$\theta = \theta - v_t$$

特点：

(1) 可以使得梯度方向不变的维度上速度变快，梯度方向有所改变的维度上的更新速度变慢，可以加快收敛并减小震荡

(2) 超参数设定值：一般 γ 取值 0.9 左右



(a) SGD without momentum



(b) SGD with momentum

■ 优化算法(Optimization Methods)

➤ Nesterov加速梯度(Nesterov Accelerated Gradient)

在计算梯度时，不是在当前位置，而是未来的位置上，

$$\begin{aligned}v_t &= \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1}) \\ \theta &= \theta - v_t\end{aligned}$$

特点：

(1) 可以使得梯度方向不变的维度上速度变快，梯度方向有所改变的维度上的更新速度变慢，可以加快收敛并减小震荡

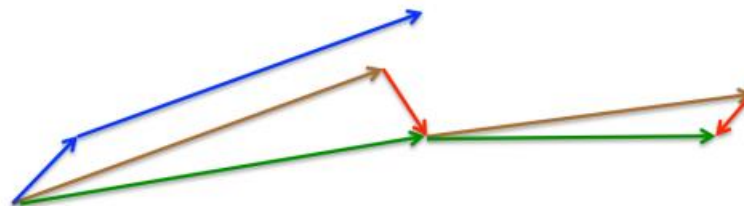
(2) 超参数设定值：一般 γ 取值 0.9 左右

■ 优化算法(Optimization Methods)

➤ Momentum+SGD 和 NAG

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$$
$$\theta = \theta - v_t$$

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta - \gamma v_{t-1})$$
$$\theta = \theta - v_t$$



- Momentum +SGD会先计算当前的梯度，然后在更新后的累积梯度后会有有一个大的跳跃(蓝线)。
- NAG 会先在前一步的累积梯度上(棕线)有一个大的跳跃，然后衡量一下梯度做修正(红线)，这种预期的更新可以避免更新太快(绿线)

■ 优化算法(Optimization Methods)

➤ 自适应梯度(Adaptive Gradient)

根据参数的重要性而对不同的参数进行不同程度的更新

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}$$

特点:

- (1) 不需要手动设置学习率更新过程
- (2) 分母会不断积累, 这样学习率就会收缩并最终会变得非常小
- (3) 超参数设定值: 一般 η 选取0.01

■ 优化算法(Optimization Methods)

➤ 自适应德尔塔(Adaptive Delta)

Adagrad的提升, 分母使用梯度的均方根

$$\Delta\theta_t = -\frac{\eta}{\sqrt{E[g^2]_t + \epsilon}}g_t \quad E[\Delta\theta^2]_t = \gamma E[\Delta\theta^2]_{t-1} + (1 - \gamma)\Delta\theta_t^2$$

$$\Delta\theta_t = -\frac{\eta}{RMS[g]_t}g_t \quad RMS[\Delta\theta]_t = \sqrt{E[\Delta\theta^2]_t + \epsilon}$$

$$\Delta\theta_t = -\frac{RMS[\Delta\theta]_{t-1}}{RMS[g]_t}g_t$$

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

特点:

- (1) 不需要手动设置学习率更新过程, 甚至不需要手动设置学习率
- (2) 缓解学习率急剧下降问题
- (3) 超参数设定值: γ 一般设定为 0.9

■ 优化算法(Optimization Methods)

➤ 梯度的均方根传播(RMSprop)

Adagrad的提升, 直接使用分母使用梯度的均方根

$$E[g^2]_t = 0.9E[g^2]_{t-1} + 0.1g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}}g_t$$

特点:

- (1) 不需要手动设置学习率更新过程
- (2) 缓解学习率急剧下降问题
- (3) 超参数设定值: γ 为 0.9, 学习率 η 为 0.001

■ 优化算法(Optimization Methods)

➤ 自适应动量估计(Adaptive Moment Estimation)

计算每个参数的自适应学习率, RMSprop + Momentum+SGD

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t & \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \end{aligned}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

特点:

- (1) Adam梯度和动量同时更新
- (2) 超参数设定值: 建议 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10e-8$
- (3) 实践表明, Adam 比其他适应性学习方法效果要好

■ 优化算法(Optimization Methods)

➤ 自适应动量估计(Adaptive Max)

针对Adam 的提升, 做Lp范数推广

$$v_t = \beta_2^p v_{t-1} + (1 - \beta_2^p) |g_t|^p \quad u_t = \beta_2^\infty v_{t-1} + (1 - \beta_2^\infty) |g_t|^\infty$$
$$= \max(\beta_2 \cdot v_{t-1}, |g_t|)$$

特点:

$$\theta_{t+1} = \theta_t - \frac{\eta}{u_t} \hat{m}_t$$

(1) 梯度和动量同时更新

(2) 超参数设定值: 建议 $\eta = 0.02$, $\beta_1 = 0.9$, $\beta_2 = 0.999$

■ 优化算法(Optimization Methods)

- 自适应动量估计((Nesterov-accelerated Adam)
针对Adam 的提升, Adam and NAG

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} (\beta_1 \hat{m}_t + \frac{(1 - \beta_1)g_t}{1 - \beta_1^t})$$

特点:

- (1) 梯度和动量同时更新
- (2) 超参数设定值: 建议 $\eta = 0.02$, $\beta_1 = 0.9$

优化算法(Optimization Methods)

➤ 可视化

鞍点

优化曲面

- 上面两种情况都可以看出，Adagrad, Adadelata, RMSprop 几乎很快就找到了正确的方向并前进，收敛速度也相当快，而其它方法要么很慢，要么走了很多弯路才找到
- 由上图可知自适应学习率方法，即 Adagrad, Adadelata, RMSprop, Adam 在这种情景下会更合适而且收敛性更好

■ 优化算法(Optimization Methods)

➤ 如何选择优化算法

如果梯度数据是稀疏的，就用自适用方法，即 **Adagrad, Adadelata, RMSprop, Adam**

RMSprop, Adadelata, Adam 在很多情况下的效果是相似的
随着梯度变的更加稀疏，**Adam** 比 **RMSprop** 效果会好

如果需要更快的收敛，或者是训练更深更复杂的神经网络，
需要用一种自适应的算法

整体来讲，Adam 是最好的选择

PART TWO

模型泛化

Model Generalization

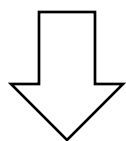


模型泛化(Model Generalization)

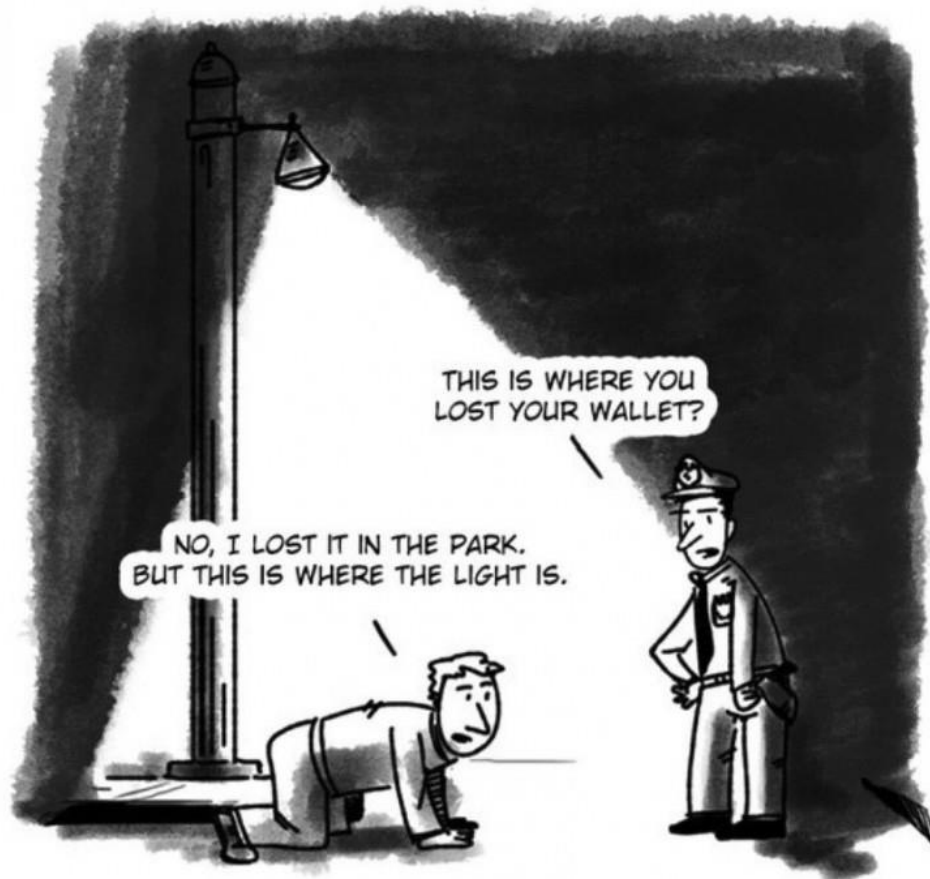
➤ 深度神经网络

过度参数化

拟合能力强



泛化性差



模型泛化(Model Generalization)

优化
经验风险最小

正则化
降低模型复杂度



模型泛化(Model Generalization)

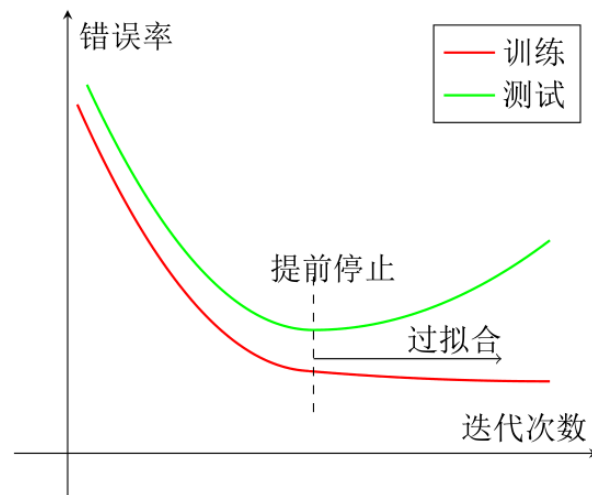
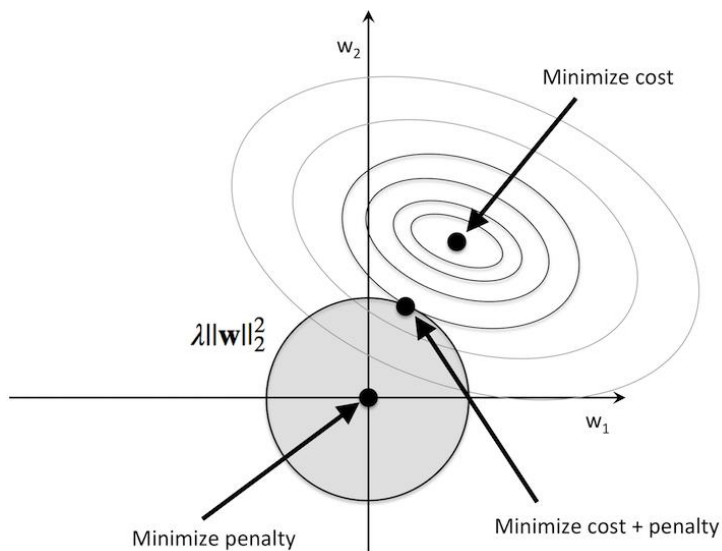
所有损害优化的方法都是正则化。

增加优化约束

干扰优化过程

L1/L2约束、数据增强

权重衰减、随机梯度下降、提前停止



模型泛化(Model Generalization)

➤ 如何提高深度神经网络的泛化能力

L1和L2正则化

早期终止

权重衰减

随机失活

数据增广

标签平滑

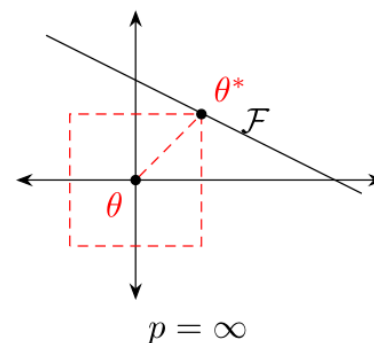
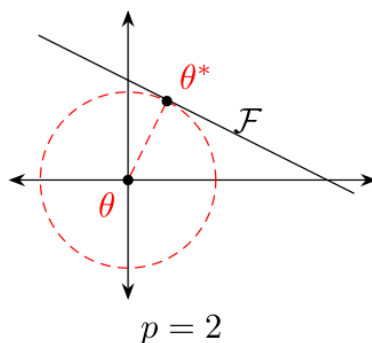
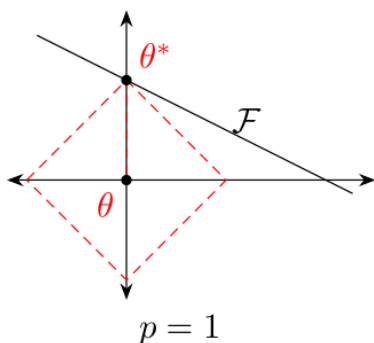
模型泛化(Model Generalization)

➤ L1和L2正则化

损失函数

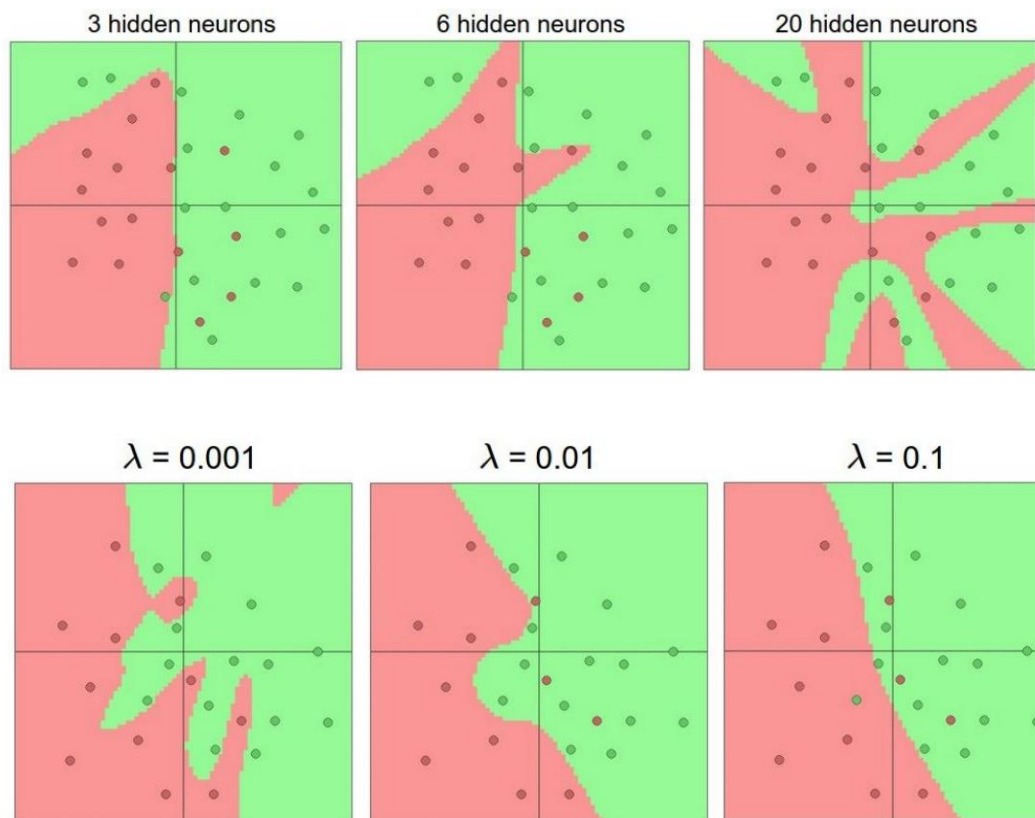
$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(\mathbf{x}^{(n)}, \theta)) + \lambda \ell_p(\theta)$$

要求权重的范数尽可能小:



模型泛化(Model Generalization)

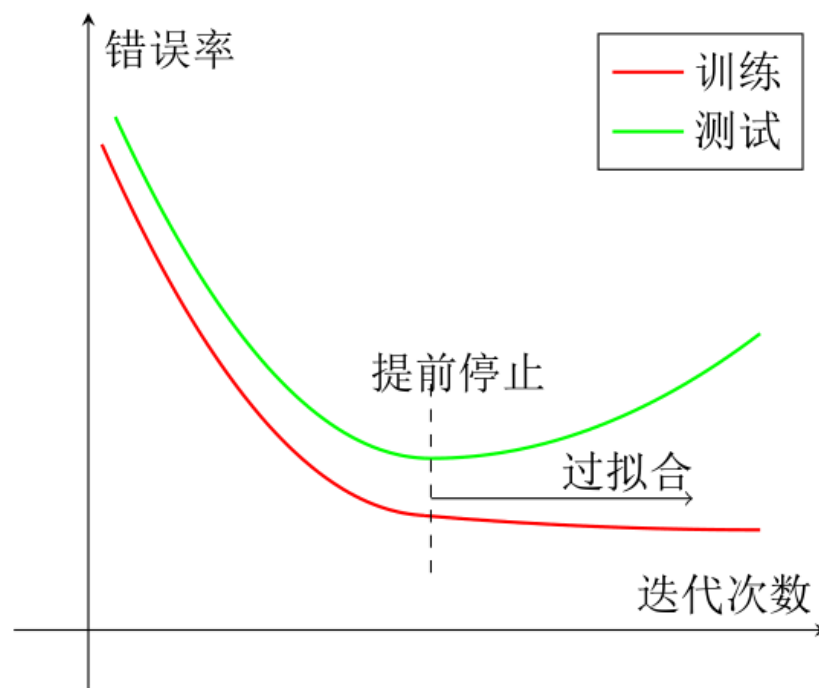
➤ L1和L2正则化



模型泛化(Model Generalization)

➤ 早期终止(Early Stop)

使用一个验证集来测试每一次迭代的参数在验证集上是否最优。如果在验证集上的错误率不再下降，就停止迭代。



■ 模型泛化(Model Generalization)

➤ 权重衰减(Weight Decay)

在每次参数更新时，引入一个衰减系数 w ，延缓更新

$$\theta_t \leftarrow (1 - w)\theta_{t-1} - \alpha g_t$$

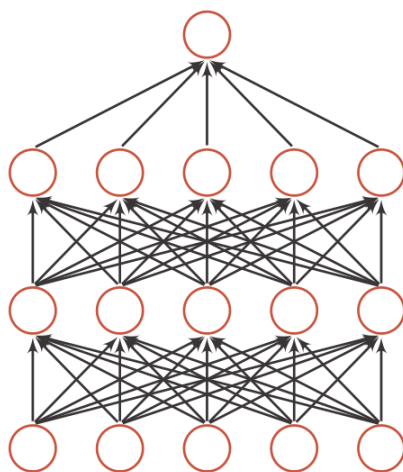
- (1) 在标准的SGD中，权重衰减正则化和L2正则化的效果等价
- (2) 在较为复杂的优化方法(比如Adam) 权重衰减正则化和L2正则化不等价

模型泛化(Model Generalization)

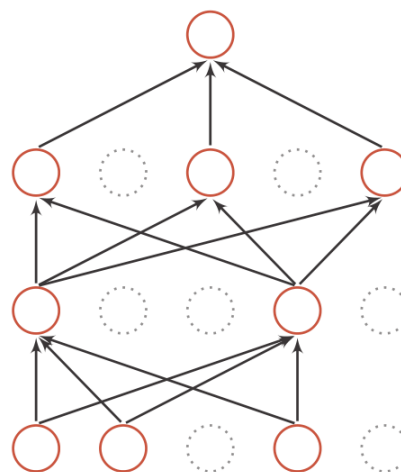
➤ 随机失活(Dropout)

在每次前向传递时，随机地将激活值置0，满足伯努利分布

$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$



(a) 标准网络



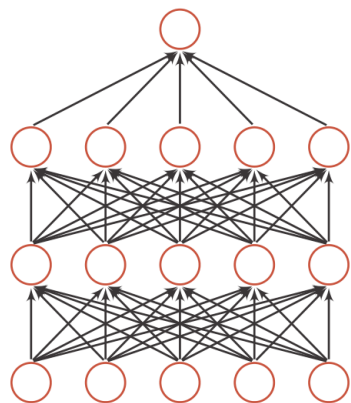
(b) Dropout 后的网络

模型泛化(Model Generalization)

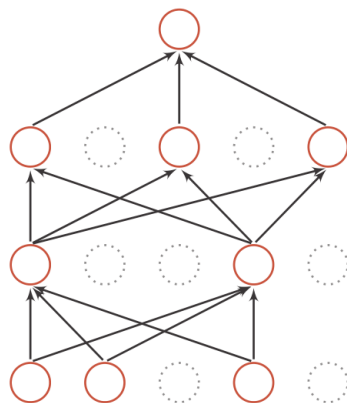
➤ 随机失活(DropOut)

在每次前向传递时，随机地将激活值置0，满足伯努利分布

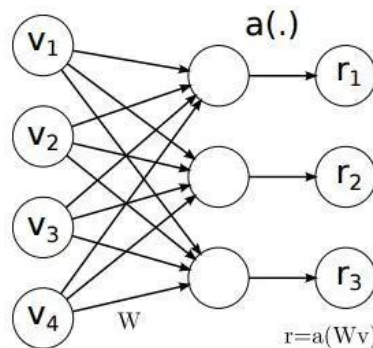
$$d(\mathbf{x}) = \begin{cases} \mathbf{m} \odot \mathbf{x} & \text{当训练阶段时} \\ p\mathbf{x} & \text{当测试阶段时} \end{cases}$$



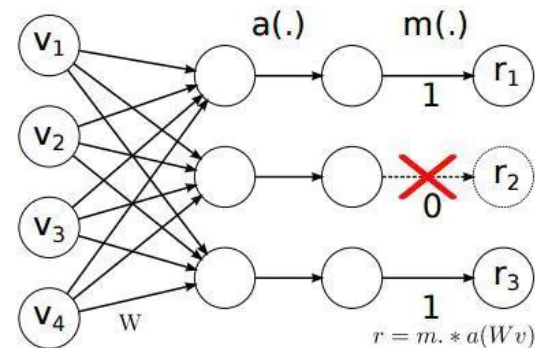
(a) 标准网络



(b) Dropout 后的网络



Normal Network



DropOut Network

■ 模型泛化(Model Generalization)

➤ 随机失活(DropOut)

集成学习的观点

每做一次失活，相当于从原始的网络中采样得到一个子网络。如果一个神经网络有n个神经元，那么总共可以采样得到 2^n 个子网络。

贝叶斯学习的观点

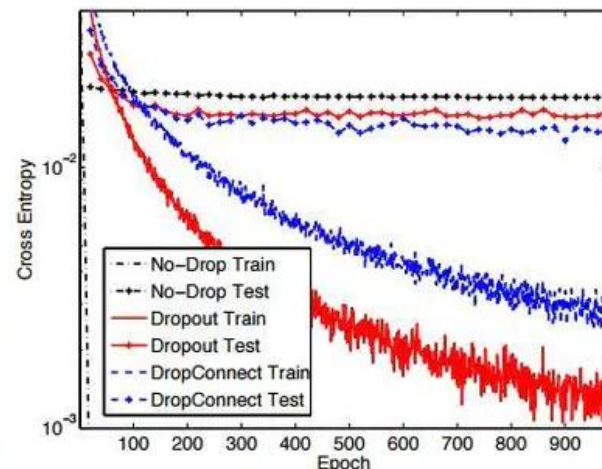
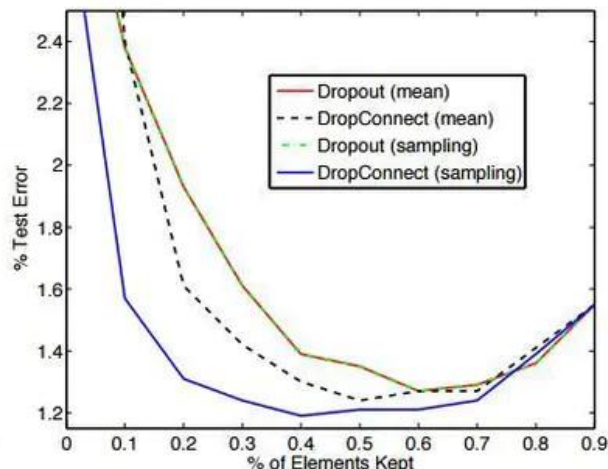
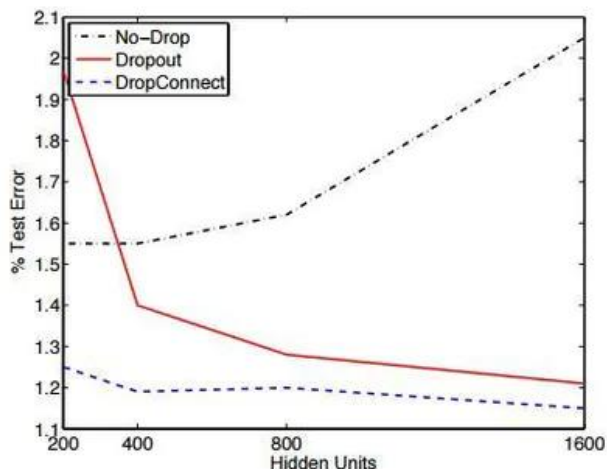
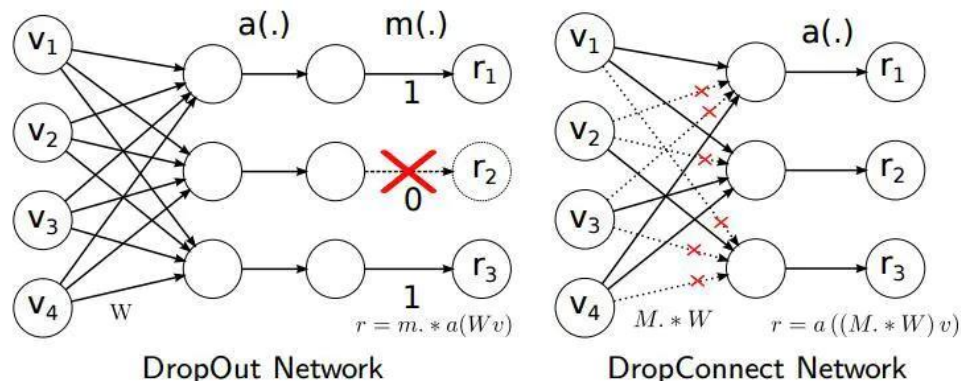
$$\begin{aligned}\mathbb{E}_{q(\theta)}[y] &= \int_q f(\mathbf{x}, \theta) q(\theta) d\theta \\ &\approx \frac{1}{M} \sum_{m=1}^M f(\mathbf{x}, \theta_m),\end{aligned}$$

其中 $f(\mathbf{x}, \theta_m)$ 为第m次应用随机失活后的网络

模型泛化(Model Generalization)

➤ 随机失连(DropConnect)

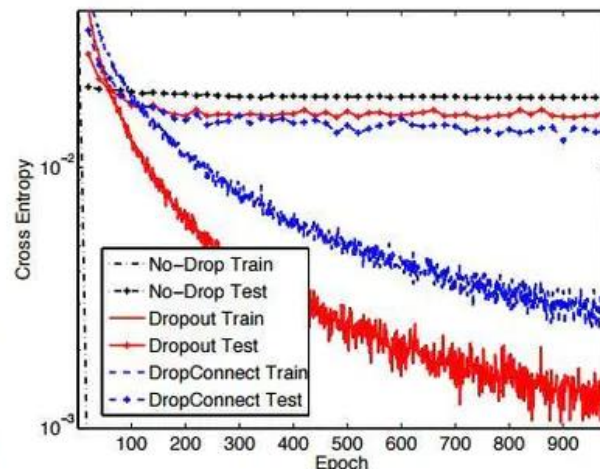
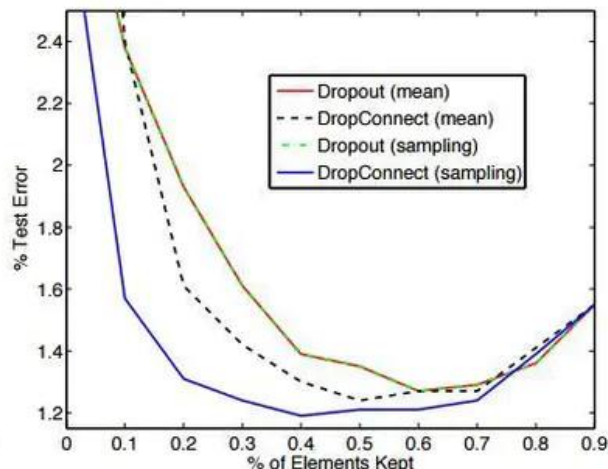
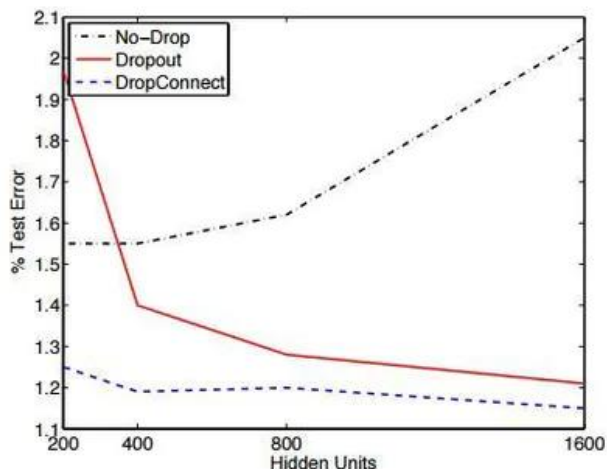
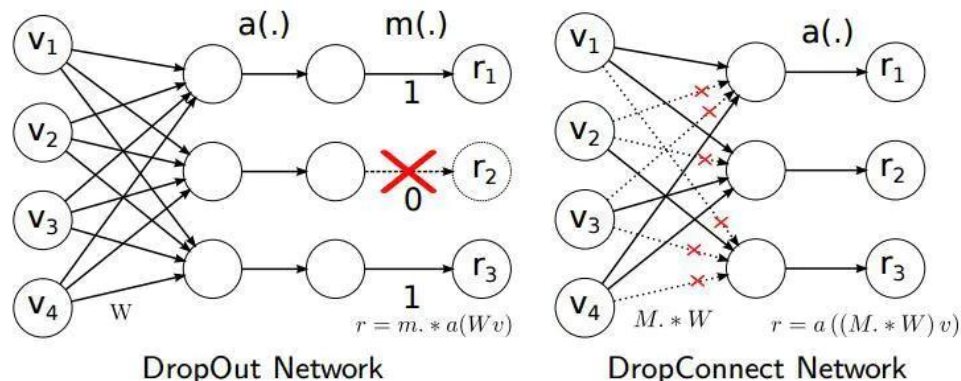
在每次前向传递时，随机地将打断网络的连接模式



模型泛化(Model Generalization)

➤ 随机失连(DropConnect)

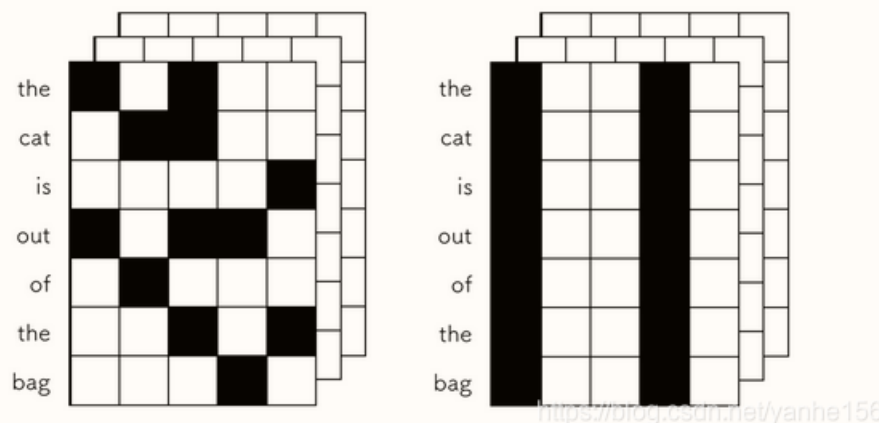
在每次前向传递时，随机地将打断网络的连接模式



■ 模型泛化(Model Generalization)

➤ 空间随机失活(Spatial DropOut)

在每次前向传递时，随机地将整个通道置0





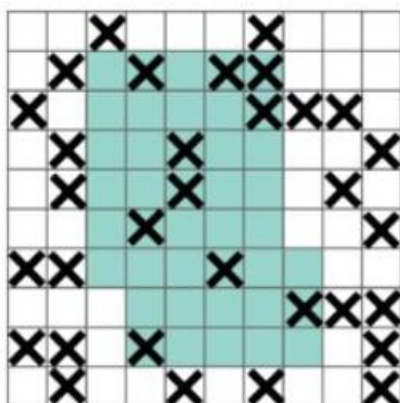
■ 模型泛化(Model Generalization)

➤ 随机失块(DropBlock)

在每次前向传递时，随机地将局部空间块置0

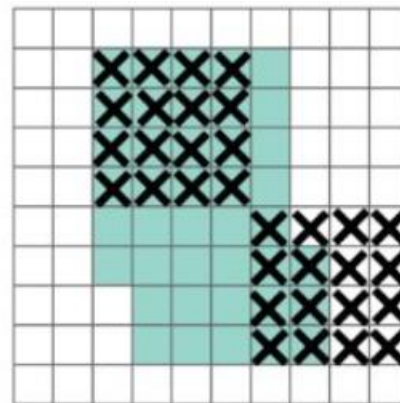


(a)



(b)

DropOut



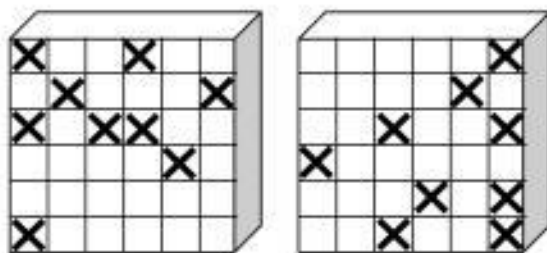
(c)

DropBlock

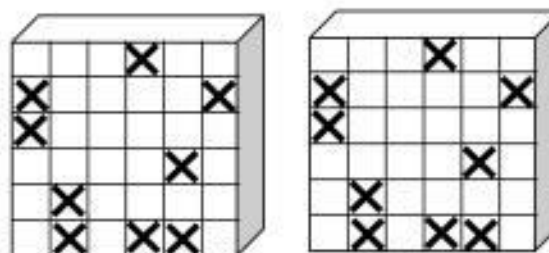
模型泛化(Model Generalization)

➤ 批次随机失块(Batch DropBlock)

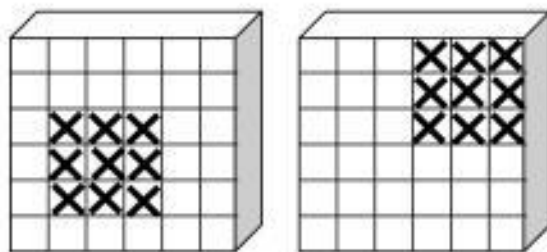
在每次前向传递时，按批次随机地将局部空间块置0



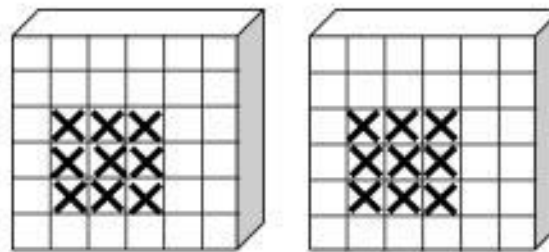
(a) Dropout



(b) Batch Dropout



(c) DropBlock



(d) Batch DropBlock

■ 模型泛化(Model Generalization)

➤ 数据增广(Data Augmentation)

主要是通过通过对图像进行变换、引入噪声等方法来增加数据的多样性。

常用的图像数据的数据增广方法：

旋转(Rotation): 按顺时针或逆时针方向随机旋转一定角度；

翻转(Flip): 将图像沿水平或垂直方法随机翻转一定角度；

缩放(Scale): 将图像放大或缩小一定比例；

平移(Shift): 将图像沿水平或垂直方法平移一定步长；

加噪声(Noise): 加入随机噪声；

聚焦(Zoom In/Out): 将图像内容放大或缩小一定比例

■ 模型泛化(Model Generalization)

➤ 标签平滑(Label Smoothing)

在输出标签中添加噪声来避免模型过拟合
一个样本 x 的标签一般用one-hot向量表示

$$\mathbf{y} = [0, \dots, 0, 1, 0, \dots, 0]^T$$

引入一个噪声对标签进行平滑，即假设样本以 ϵ 的概率为其它类。平滑后的标签为

$$\tilde{\mathbf{y}} = [\frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}, 1-\epsilon, \frac{\epsilon}{K-1}, \dots, \frac{\epsilon}{K-1}]^T$$



PART THREE

小结

Summary



■ 小结

➤ 迭代算法

- ✓ 批次梯度下降
- ✓ 随机梯度下降
- ✓ 小批次梯度下降
- ✓ **Momentum+SGD**
- ✓ **NAG**
- ✓ **Adagrad**
- ✓ **Adadelata**
- ✓ **RMSprop**
- ✓ **Adam, AdaMax, Nadam**

➤ 模型泛化

- ✓ **L1和L2正则化, 权重衰减**
- ✓ **早期终止**
- ✓ **随机失活**
- ✓ **数据增广**
- ✓ **标签平滑**

